EXPERIMENT 9

ROLL NO: 240701161 NAME: HANISHKHA N

Design input forms that validate data (e.g., email, phone number) and display error messages using HTML/CSS, JavaScript and Each and every step should have an output screen with an explanation

AIM: The aim is to design input forms that validate data, such as email and phone number, and display error messages using HTML/CSS and JavaScript with Validator.js.

Step 1:

Setting Up the HTML Form Start by creating an HTML form with input fields for the email and phone number

Explanation:

The first step in designing a validated input form is to establish the **Document Object Model (DOM)** structure using HTML5. In this experiment, the form is designed with accessibility and validation in mind.

Key Components of the HTML Structure:

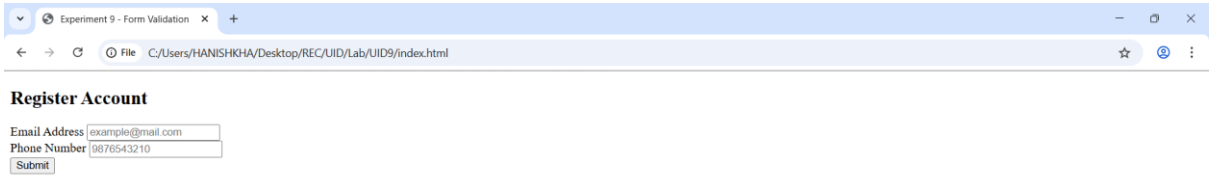
- **Form Container (<form>):** Encapsulates the input fields and the submit button. It is assigned a unique id="registrationForm" so that JavaScript can easily target it for event handling.
- **Semantic Inputs:** We use <input type="text"> for both the email and phone number. While HTML5 offers type="email", we are using text here to demonstrate how to perform manual, robust validation using the **Validator.js** library.
- **Placeholder Attributes:** These provide a visual hint to the user regarding the expected data format (e.g., example@mail.com).
- **External Library Integration:** A <script> tag is included at the bottom to link the **Validator.js CDN**, which provides the necessary functions for checking data integrity.

Code:

```
<!DOCTYPE html>

<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Experiment 9 - Form Validation</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h2>Register Account</h2>
        <form id="registrationForm">
            <div class="input-group">
                <label for="email">Email Address</label>
                <input type="text" id="email" placeholder="example@mail.com">
                <span id="emailError" class="error"></span>
            </div>

            <div class="input-group">
                <label for="phone">Phone Number</label>
                <input type="text" id="phone" placeholder="9876543210">
                <span id="phoneError" class="error"></span>
            </div>

            <button type="submit">Submit</button>
        </form>
    </div>
```

```
    <script
src="https://unpkg.com/validator@latest/validator.min.js"></script>
    <script src="script.js"></script>
</body>
</html>
```



The output screen shows the basic skeletal structure of the web page. At this stage, the elements are un styled and follow the default browser alignment. It consists of a heading "Register Account," two labelled input fields with placeholder text, and a standard submit button.

Step 2:

Styling the Form with CSS Next, add some basic styling to make the form look nice.

Explanation:

In this step, Cascading Style Sheets (CSS) are used to transform the basic HTML structure into an aesthetically pleasing and user-friendly interface. The styling focuses on layout, spacing, and visual feedback.

Key Design Elements:

- **Flexbox Centering:** The body uses display: flex with justify-content and align-items set to center. This ensures the form is perfectly positioned in the middle of the viewport regardless of screen size.
- **Box-Model & Card Layout:** The .container is styled with a white background, specific padding, and border-radius to create a modern "card" effect. A subtle box-shadow is applied to give the form depth and separate it from the background.
- **Typography & Form Elements:** Custom fonts and consistent margins are applied to labels and inputs to improve readability. The input fields are given a 100% width and specific padding to make them touch-friendly and visually balanced.
- **Interactive Styling:** The submit button is styled with a distinct background color (#4A90E2) and a transition property. This provides a smooth visual change when the user hovers over it, enhancing the **User Experience (UX)**.
- **Error Message Visibility:** The .error class is styled with a specific color (red) and font size. By defining its position in CSS, we ensure that when JavaScript injects text, the layout remains stable without shifting other elements abruptly.

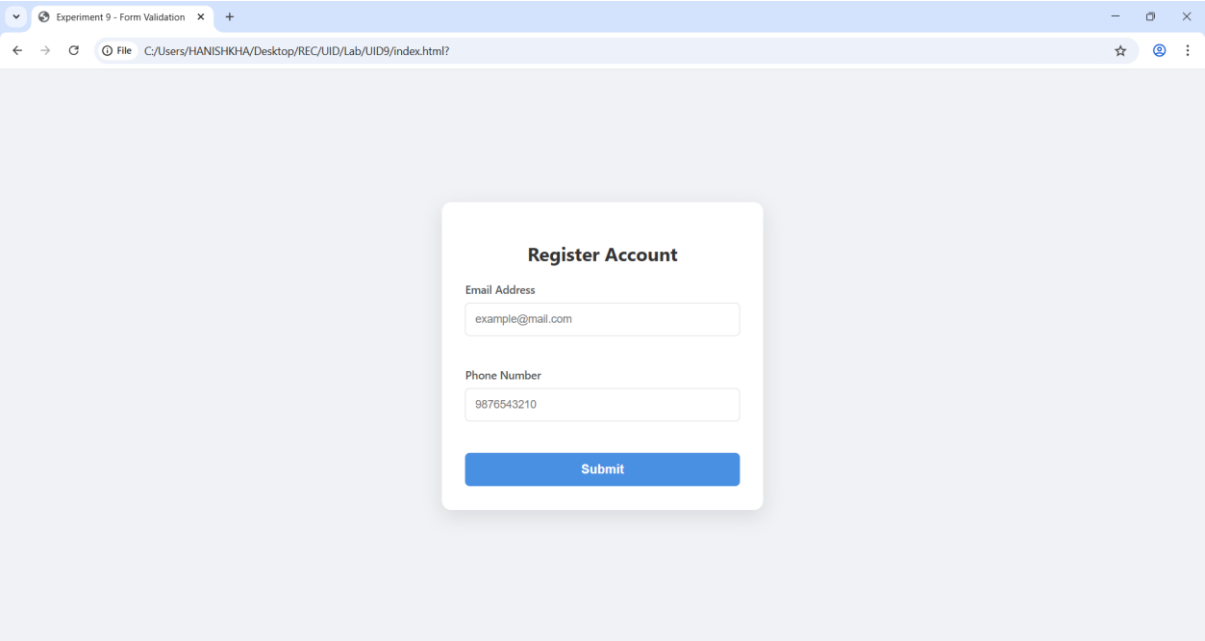
Code:

```
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f0f2f5;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
}


.container {
```

```
    background: #ffffff;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 8px 24px rgba(0,0,0,0.1);
    width: 350px;
}

h2 {
    text-align: center;
    color: #333;
    margin-bottom: 20px;
}

.input-group {
    margin-bottom: 20px;
}

label {
    display: block;
    margin-bottom: 8px;
    font-size: 14px;
    font-weight: 600;
    color: #555;
}

input {
    width: 100%;
```

```
    padding: 12px;
    border: 1.5px solid #ddd;
    border-radius: 6px;
    font-size: 14px;
    transition: border-color 0.3s;
    box-sizing: border-box;
}

input:focus {
    outline: none;
    border-color: #4A90E2;
}

.error {
    color: #e74c3c;
    font-size: 12px;
    margin-top: 5px;
    display: block;
    min-height: 15px; /* Prevents layout jump when error appears */
}

button {
    width: 100%;
    padding: 12px;
    background-color: #4A90E2;
    color: white;
    border: none;
```

```
    border-radius: 6px;
    font-size: 16px;
    font-weight: bold;
    cursor: pointer;
    transition: background 0.3s;
}

button:hover {
    background-color: #357ABD;
}
```



The output screen shows a professional, centred registration card. The form now features a clean white container with rounded corners and a soft shadow against a light grey background. The input fields are neatly aligned, and the blue "Submit" button stands out as the primary **Call to Action (CTA)**.

Step 3:

Adding JavaScript for Validation Finally, add JavaScript to validate the input fields using Validator.js and display error messages.

Explanation:

This final step involves implementing the functional logic using **JavaScript** and the **Validator.js** library. The script acts as the "brain" of the form, intercepting the submission process to ensure data integrity before any further action is taken.

Key Technical Processes:

- **Event Handling:** An event listener is attached to the submit event of the form. Using event.preventDefault(), we stop the browser's default behavior (reloading the page), allowing our script to process the data locally.
- **Data Extraction & Cleaning:** The script retrieves values from the email and phone input fields and uses .trim() to remove any accidental white spaces that might cause validation to fail.
- **Library Logic Implementation:** * **validator.isEmail()**: Scans the input string for a valid structure (name, @ symbol, and domain).
    - **validator.isMobilePhone()**: Checks if the numeric string matches standard mobile formatting.
- **Success Notification:** Only if both validation checks return true does the script trigger a browser alert, notifying the user that the data is valid and ready for backend processing.

Code:

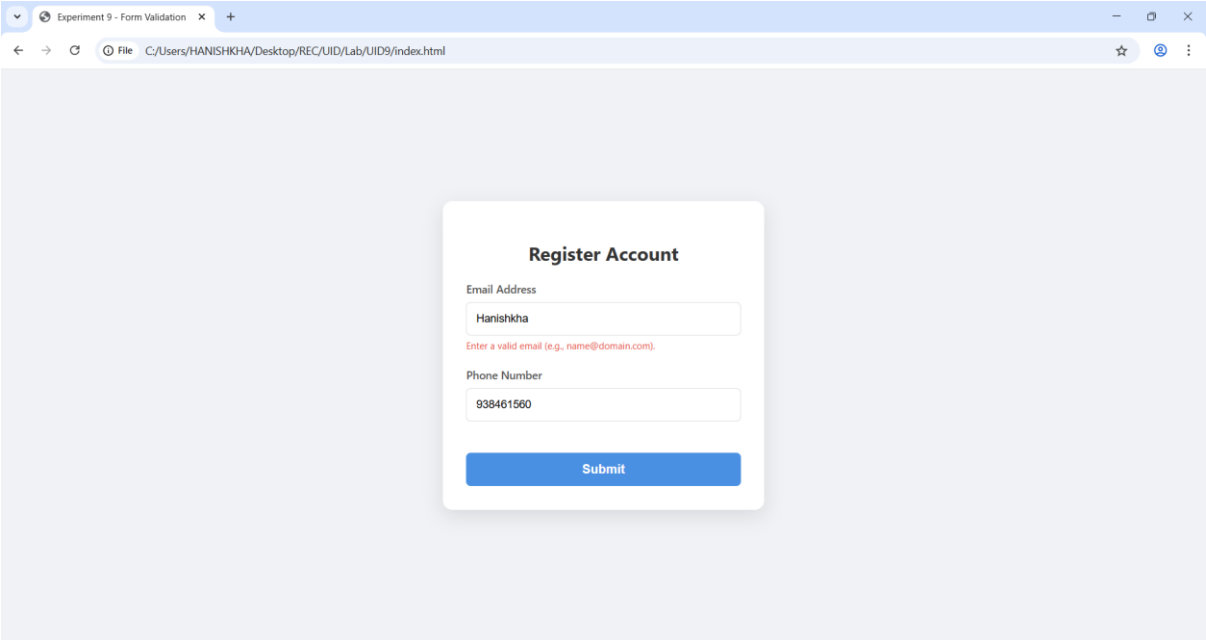
```
document.getElementById('registrationForm').addEventListener('submit',
function(event) {

    // 1. Prevent the form from refreshing the page

    event.preventDefault();

    // 2. Grab the values from the input fields

    const emailValue = document.getElementById('email').value.trim();

    const phoneValue = document.getElementById('phone').value.trim();

    // 3. Select error message placeholders

    const emailError = document.getElementById('emailError');

    const phoneError = document.getElementById('phoneError');
```

```
    // 4. Clear any previous error messages
    emailError.textContent = "";
    phoneError.textContent = "";
    let isFormValid = true;
    // 5. Email Validation: Use Validator.js to check format
    if (validator.isEmpty(emailValue)) {
        emailError.textContent = "Email is required.";
        isFormValid = false;
    } else if (!validator.isEmail(emailValue)) {
        emailError.textContent = "Enter a valid email (e.g.,
name@domain.com).";
        isFormValid = false;
    }

    // 6. Phone Validation: Check for a valid mobile number
    if (validator.isEmpty(phoneValue)) {
        phoneError.textContent = "Phone number is required.";
        isFormValid = false;
    } else if (!validator.isMobilePhone(phoneValue, 'any')) {
        phoneError.textContent = "Enter a valid mobile phone number.";
        isFormValid = false;
    }

    // 7. Success Action
    if (isFormValid) {
        alert("Success! Form data is valid and ready to be sent.");
        // Here you could add code to send data to a server
```

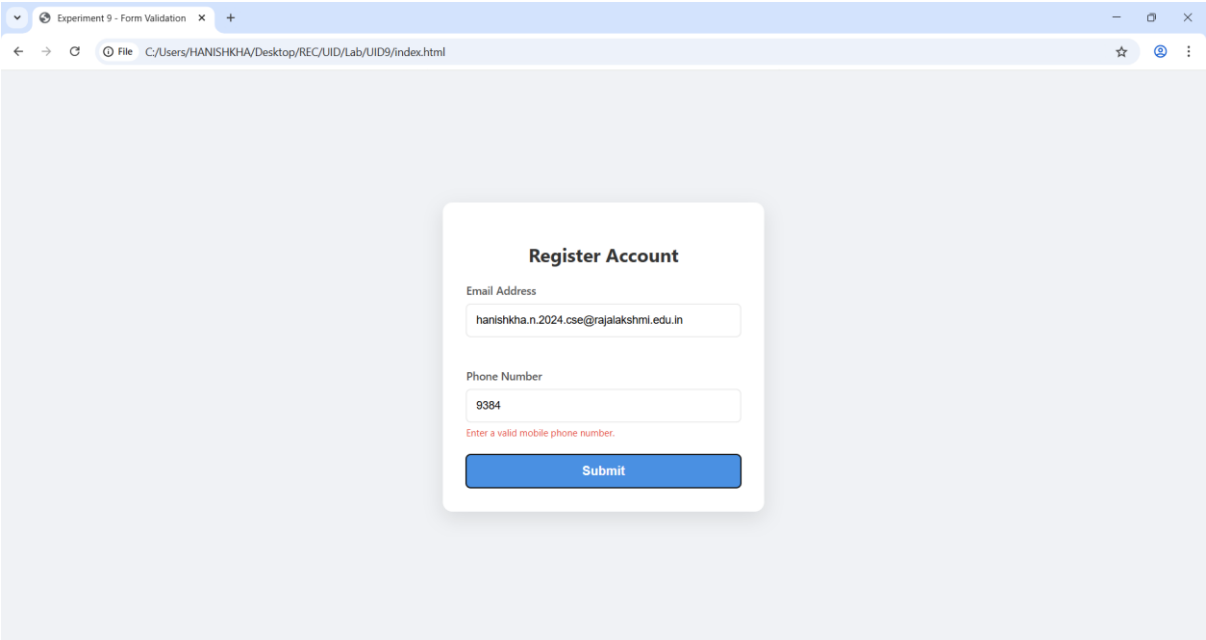
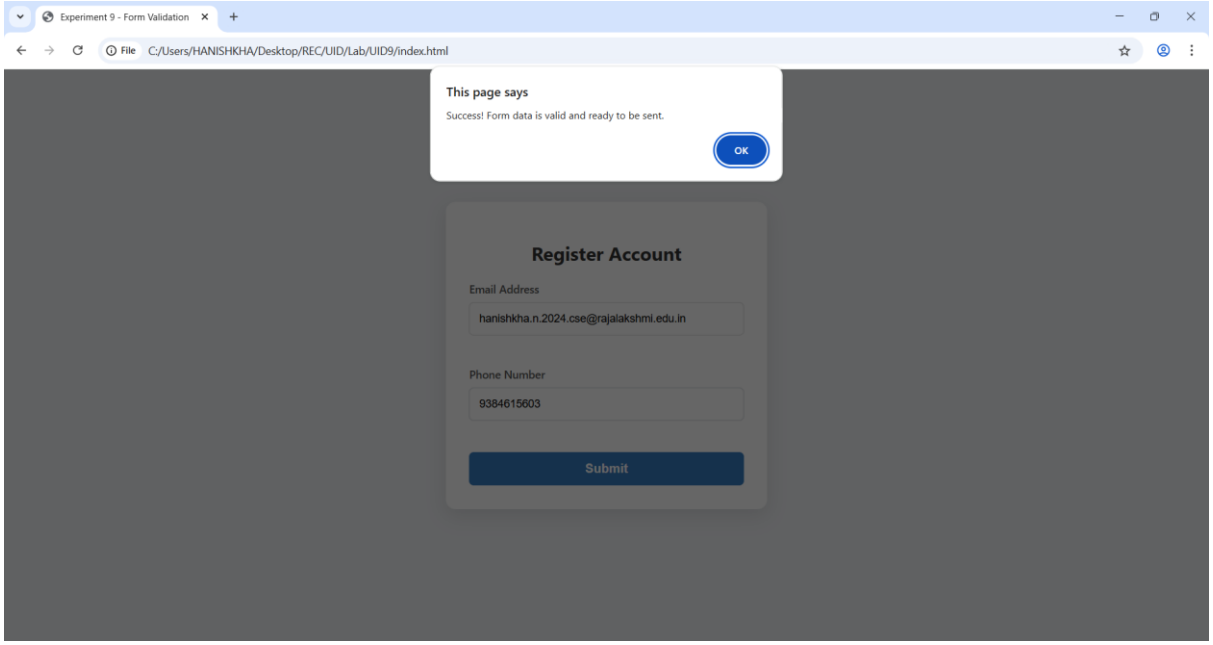
```
    }
});
```

The output is demonstrated through three scenarios captured in the screenshots:

1. **Email Validation Error:** When "Hanishkha" is entered without an @ or domain, the script displays the red error message: *"Enter a valid email (e.g., name@domain.com)."*



2. **Phone Validation Error:** When an incomplete number "9384" is entered, the script displays: *"Enter a valid mobile phone number."*



3. **Success State:** When correct data is entered (as seen with your REC email ID), a pop-up alert confirms: *"Success! Form data is valid and ready to be sent."*